

Type Safety via Hoare Logic with Separation Type

Wenhua Li Darius Foo Quang-Trung Ta Wei-Ngan Chin

Dept of Computer Science, National University of Singapore

Abstract. Traditionally, type safety for user programs is ensured by crafted type systems that embrace the motto “*well-typed programs cannot go wrong*” at runtime due to type errors. However, we are increasingly reliant on type systems for more advanced considerations, such as greater memory safety (e.g., Rust), and the need to support dynamically-typed languages (e.g., Python). In this paper, we show how Floyd-Hoare logic with a novel combination of *separation* types (inspired by separation logic) and *pure* types (from traditional flow-insensitive types) can provide a unified and foundational framework to meet these new demands. Our new proposal has several benefits. Firstly, we can flexibly handle type safety for different paradigms, from *statically typed languages* (such as ML) to *dynamically typed languages* (such as Python) via semantic subtyping. Secondly, we can naturally support *path-sensitive* and *flow-sensitive* type specifications. Lastly, we give a comprehensive set of error types for different scenarios, (i) **Err** type to capture *runtime errors* as values that can be passed into arguments of method calls; (ii) **Abrt** type to denote *compile-time errors* that would have triggered program abortions and must be flagged as type errors; and (iii) **Exc** type that can be used to support *checked* and *unchecked* exceptions. In a nutshell, our work greatly increases coverage on *type safety*, and we support *tractability* by limiting to semantic subtyping over a Boolean algebra of types.

1 Introduction

In recent years, there has been an increasing emphasis on designing stronger type systems for programming languages. Amongst statically-typed languages, we are seeing ever more advanced typing features being deployed, such as (i) GADTs (generalized algebraic data types with type constraints) in OCaml, Scala and Haskell, (ii) more precise subtyping (for wider typeability) from MLsub [10] and MLstruct [25], and (iii) the use of modal types [24] (to support low-latency codes) for OCaml [29]. Similarly, dynamically-typed languages (e.g. JavaScript) are being retrofitted with gradually-typed systems (e.g. TypeScript).

In this paper, we propose a foundational approach, based on Hoare logic, to perform *modular* type safety verification. We first illustrate how the type signature of a function can be represented as a pre/post *type specification*. We then show how the Hoare logic framework (with separation type) can help us achieve better *modularity* and *precision* in support of greater type safety guarantees.

Consider three functions from an ML-like language, namely a (+) function to add two integers together, and a polymorphic (!) function to dereference a mutable **Ref** data type and a higher-order **apply** function.

```

(+)    : (Int, Int) → Int
(!)    : ∀T. Ref(T) → T
apply  : ∀A,B. (A→B) → A → B
apply f x = f x

```

While type signatures are rather concise, it is almost as easy to write their type specifications using pre/post conditions in Hoare logic, as shown below.

```

(+)    : req[x,y] x:Int ∧ y:Int ens[r] r:Int
(!)    : ∀T. req[x] x:Ref(T) ens[r] r:T
apply  : ∀A,B. req[f] f:(A→B) (req[x] x:A ens[r] r:B)

```

In our version of pre/post type specification of the form $\text{req}[x] x:t_1 \text{ ens}[r] r:t_2$, each **req** clause captures a function parameter x and its type t_1 , while the **ens** clause gives a name r for the function’s result and its type t_2 . We can write the type signature $t_1 \rightarrow t_2$ as an abbreviation. However, type signatures do not capture global variables and are also flow-insensitive. The latter problem is especially challenging for dynamic languages. A simple example is shown below where `str_of_int` converts an `Int` into its `Str` form.

```

stateful_add : Int → Str
stateful_add y = update(g, str_of_int(y+!g)); !g

```

The `update(m, v)` function allows to mutate the value stored in m and change it to v . The `update` function is discussed later in Sec. 2.2. For accurate tracking of the types of some memory locations, such as g , where types can be mutated, we must provide for a flow-sensitive type system. In this paper, we introduce *separation type* of the form $g \rightarrow \text{Ref}(t_1)$ that is flow-sensitive, in that we allow program codes to change their type to a different $g \rightarrow \text{Ref}(t_2)$. This allows us to write a more accurate type specification for `stateful_add`:

```

stateful_add : req[x] x:Int ∧ g→Ref(Int)
               ens[r] r:Str ∧ g→Ref(Str)

```

While flow-insensitive pure types, such as $g:\text{Ref}(\text{Any})$, might also have been used, such use leads to weaker type-safety guarantees. In this paper, we develop a Hoare logic framework for type-safety where the flow-sensitivity of types can be supported by *separation types* (with must-aliasing) together with flow-insensitive *pure types* (with arbitrary-aliasing) whenever type-mutation is not required.

Our main technical contributions are:

- We advocate for a Floyd-Hoare logic approach to type-safety verification.
- We introduce a novel type, called *separation type*, that can support must aliasing and type mutation, and thus flow-sensitivity on types. This is used in conjunction with *pure types* that are traditionally flow-insensitive. These two kinds of types provide a framework for increased coverage on type safety.
- Our approach can support both strongly-typed languages with advanced types; and dynamically-typed languages with semantic-based subtyping.
- We provide a set of four error types for comprehensive error handling, with `Abrt`-type, flagged as *serious* compile-time error if it is statically reachable.

- We design a small core language, define its operational semantics, and construct a logic of separation and pure types. We also formalize our Hoare rules in Rocq, and show how foundational type-safety proofs are constructed.
- We prototype our type specification framework into a verifier called Heifer-type and evaluate its capabilities on a set of benchmark programs.

Paper structure. The rest of our paper is organized as follows. Sec 2 covers novel aspects of our new type logic via examples. Sec 3 presents a core language and a logic for types that support expressive type specifications for both statically-typed and dynamically-typed languages. Sec 4 presents a set of forward-style Hoare rules to ensure type safety. Sec 5 presents the semantics of our logic on types and formalization of our Hoare rules in Rocq, and describes a prototype verifier implemented in OCaml. Appendix covers more features of our solution.

2 Improvements to Type-Safety

We present three novel aspects of our Hoare-logic approach to type-checking. We first show how semantic-based subtyping can be generalized to a logic of types. We then introduce flow-sensitive *separation types* and show how they are related to *pure types*. Lastly, we design a new hierarchy of types to cover both separation and pure types with a comprehensive classification of error types.

2.1 Semantic-Based Subtyping via Hoare Logic

Semantic-based subtyping interprets each type t as a set of values, and then defines subtyping $t_1 <: t_2$ using the set containment relation $\text{Set}(t_1) \subseteq \text{Set}(t_2)$. With this, one could rely on a Boolean algebra of types to more precisely and flexibly compose types, as shown below.

$$t := t \vee t \mid t \wedge t \mid \neg t \mid \dots$$

By using union, intersection, and negation of types, we can support more expressive types directly. Consider the ML-style `inc_if_int` function below:

```
(* Implementation of inc_if_int *)
inc_if_int(x) = match x of { Int -> x+1;
                           _   -> "Not supported!" }
```

Existing works. ML's type system would reject this program, since it uses unification that fails when it combines the `Int` type from one branch with the `Str` type from another. In contrast, semantic subtyping can unify the output type as `Int ∨ Str`, that takes into account both branches of the above method, with the help of its more expressive Boolean algebra of types.

```
(* Type signature of inc_if_int *)
inc_if_int : Any → Int ∨ Str
```

Our proposal. We propose to extend semantic subtyping into a logic of types under Hoare-style reasoning. With this, we can freely use our logic of types (with semantic subtypes) to develop stronger type-safety guarantees. Thus, under Hoare logic, we can now express relational type-properties between a set of inputs and/or outputs. Our earlier example can be expressed precisely as:

```
(* Type specification of inc_if_int *)
inc_if_int : req[x] x:Any ens[r] (x:Int∧r:Int)∨(x:¬Int∧r:Str)
```

where relational type properties between inputs and/or outputs of methods can be captured via disjunction, leading to type specifications that are path-sensitive.

Standard type systems may achieve path-sensitivity by using intersection types, typically written in the form $(t_1 \rightarrow t_2) \wedge (t_3 \rightarrow t_4)$. Here, t_1 and t_3 are expected to be of disjoint types, leading to possibly different output types t_2 and t_4 , respectively. Thus, one may use the following intersection type.

```
(* Standard type signature using intersection type *)
inc_if_int : (Int → Int) ∧ (¬Int → Str)
```

Our preference is case specification, as shown below for `inc_if_int`. Unlike intersection types, each case specification is forced to use disjoint case analysis.

```
(* Our proposed type specification using case-specs *)
inc_if_int : case[x] { x:Int ⇒ ens[r] r:Int;
                      x:¬Int ⇒ ens[r] r:Str }
```

As presented in [19], the case construct can be adopted in our type specifications to achieve better completeness for its proving (or checking) procedure. In path-sensitive analysis, case specifications are more informative as they clearly state that the differences in outputs are caused by the differences in x 's types. When properly designed, case specifications could guide programmers in understanding their programs and whether they have considered all scenarios.

2.2 Flow-Sensitivity and Separation Types

Many languages support pass-by-value-result parameters (e.g. `inout` parameters for Swift) that allow the variable arguments at such positions to be mutated by the corresponding method calls. Additionally, dynamically-typed languages allow the types of their variables to be arbitrarily changed, as shown by the Swift-style method below where `str_of_int` converts an `Int` into its `Str`.

```
inc_transform (inout x) = x := str_of_int(x+1)
```

Existing works. One may suggest $\text{Int} \vee \text{Str} \rightarrow ()$ as the type signature of `inc_transform`, but this is still not type-safe since $\text{Int} \vee \text{Str}$ is weaker than the `Int` pre-condition required for the parameters of `(+)` method. The problem here is that traditional type systems (including Swift's) are typically *flow-insensitive*.

Our proposal. Hoare logic can easily solve this problem since the pre/post specification can capture type changes. An example of this is shown below.

```
inc_transform : req[x] x:Int ens[r] x':Str ∧ r:()
```

where x' in the post-condition is used to denote the changed type of parameter x . This occurs in the above type specification with the mutated type $x':\text{Str}$.

Tracking type mutation for heap memory locations is considerably trickier since we have to take into account must aliases that could occur amongst heap memory locations. One way of solving this problem is to use *uniqueness type*

(from linear types) which can guarantee that each such location to heap memory has only a single reference and no other aliases. This solution is currently deployed in OCaml type system[24], but requires supporting mechanisms, such as locality and use-once analysis (for higher-order functions). Prior to linear types, the earliest flow-sensitive type system is based on the notion of *typestate* [30] that is used to track the abstract state of an object over its lifetime.

In this paper, we propose a new resource type, called *separation type*, which is inspired by separation logic. To illustrate the need for this novel type, consider the type specifications for a heap-allocated but mutable **Ref** data type.

```
mkRef :  $\forall T: \text{Any}_P. \text{req}[x] \ x: T \ \text{ens}[r] \ r: \text{Ref}(T)$ 
(!)    :  $\forall T: \text{Any}_P. \text{req}[m] \ m: \text{Ref}(T) \ \text{ens}[r] \ r: T$ 
update:  $\forall T: \text{Any}_P. \text{req}[m, v] \ m: \text{Ref}(T) \wedge v: T \ \text{ens}[r] \ r: ()$ 
```

The above type specifications are flow-insensitive since each $m: \text{Ref}(T)$ is currently a flow-insensitive pure type and Any_P denotes the set of all pure types. Such flow-insensitive types may be fine for immutable data structures and read-only methods, like $(!)$, but causes weaker specifications for the **mkRef** and **update** methods. (An implementation for the **Ref** library is described in App A.4.)

We introduce *separation type*, denoted by $m \mapsto \text{Ref}(T)$, as a resource type in our logic to denote each heap location, at address m , that can be mutated. We also provide spatial conjunction $*$ to capture a pair of disjoint heap locations, e.g. $m_1 \mapsto \text{Ref}(t_1) * m_2 \mapsto \text{Ref}(t_2)$, that could be separately mutated. Using such *separation types*, we can provide stronger flow-sensitive specifications, as follows.

```
mkRef: req[x] x: Any ens[r] r  $\mapsto$  Ref(x)
(!):  $\forall T: \text{Any}_P, q: \text{Any}. \text{case}[m] \{ m: \text{Ref}(T) \Rightarrow \text{ens}[r] \ r: T; \\ m \mapsto \text{Ref}(q) \Rightarrow \text{ens}[r] \ m \mapsto \text{Ref}(q) \wedge r: q \}$ 
update:  $\forall T: \text{Any}_P. \text{case}[m, v] \{ m: \text{Ref}(T) \wedge v: T \Rightarrow \text{ens}[r] \ r: (); \\ m \mapsto \text{Ref}(\_) \wedge v: \text{Any} \Rightarrow \text{ens}[r] \ m \mapsto \text{Ref}(v) \wedge r: () \}$ 
```

The pre-condition of **mkRef** is the weakest possible, with Any denoting either pure or separation types. The post-condition of **mkRef** is now stronger since $r \mapsto \text{Ref}(T)$ with pure type T can be irreversibly weakened to $r: \text{Ref}(T)$, if desired. Moreover, the type specifications of **update** and $(!)$ now have extra pre/post specifications using separation types, that are potentially flow-sensitive. Each separation type $m \mapsto \text{Ref}(q)$ may either have pure component, as denoted by $m \mapsto \text{Ref}(q: T) \equiv m \mapsto \text{Ref}(q) \wedge q: T$, or separation component, as denoted by $m \mapsto \text{Ref}(q \mapsto T) \equiv m \mapsto \text{Ref}(q) * q \mapsto T$. These scenarios allow for more expressive type specifications.

With separation types, we may now support either programmer-managed memory (e.g., C) or compiler-managed memory (e.g., Rust or OCaml). In the presence of (flow-insensitive) pure types, we would also require garbage collection for heap recovery (e.g., Haskell). Explicit deallocation of heap locations can be carried out using a **free** method with the following type specification.

```
free: req[m] m  $\mapsto$  Ref(_) ens[r] m: Dead  $\wedge$  r: ()
```

On each successful **free**, the separation type $m \mapsto \text{Ref}(_)$ would be mutated to $m: \text{Dead}$ denoting a dangling reference. Each instance of $m: \text{Dead}$ is considered a

run-time error value, namely $\text{Dead} <: \text{Err}$, that must be explicitly tracked by our type system. This ensures that dangling references are never wrongly accessed. This use of separation types also applies to immutable data constructors as well. Memory of $\text{x} \mapsto \text{List}(\text{T})$ can be similarly freed and reused at compile-time, while that of pure type $\text{x} : \text{List}(\text{T})$ would have to be garbage-collected. For simplicity, we require programmers to manually insert **free** commands, though it should be possible to support automatic insertion, as guided by the use of **Dead** types.

We also provide a weakening lemma that allows each separation type to be irreversibly weakened to its corresponding pure type, as shown below.

$$\text{x} \mapsto \text{T} \wedge \text{T} <: \text{Any}_P \wedge \text{T} <: \text{A} \Rightarrow_{\text{weaken}} \text{x} : \text{A}$$

This weakening lemma ensures that each separation type could contain pure types as sub-components, but not vice-versa. For example, $\text{x} \mapsto \text{Ref}(\text{Ref}(\text{T}))$ is allowed but $\text{x} : \text{Ref}(\text{q} \mapsto \text{Ref}(\text{T}))$ is disallowed. This is so as full ownership of heap locations cannot be guaranteed via pure types. We have incorporated this weakening lemma into our automatic proving procedure, as outlined in App D.4.

Thus, we always start with types that are as strong as possible, but allow them to be weakened, when necessary. We also start with separation type when the data structures are first constructed, but allow them to be irreversibly weakened to flow-insensitive pure-type, when type mutation is no longer required.

To illustrate the need for aliasing, consider a **swap** method to exchange the contents of two heap locations, as shown below.

```
swap(x, y) = let v1 = !x in let v2 = !y in
             update(x, v2); update(y, v1)
```

With separation types, we can support at least three scenarios by using the following type case specification.

```
swap : ∀A : Any_P, a, b : Any. case [x, y] {
  x : Ref(A) ∧ y : Ref(A) ⇒ ens[r] r : ();
  x ↦ Ref(a) * y ↦ Ref(b) ⇒ ens[r] x ↦ Ref(b) * y ↦ Ref(a) ∧ r : ();
  x ↦ Ref(a) ∧ y : x ⇒ ens[r] x ↦ Ref(a) ∧ r : () }
```

The first case handles pure types $\text{x} : \text{Ref}(\text{A}) \wedge \text{y} : \text{Ref}(\text{A})$, but it does not allow type mutation. The second case is used to support $\text{x} \mapsto \text{Ref}(\text{a}) * \text{y} \mapsto \text{Ref}(\text{b})$ with two distinct references (x, y) whose values (denoted by **Any** type) are swapped, while the third specification is used to support $\text{x} \mapsto \text{Ref}(\text{a}) \wedge \text{y} : \text{x}$ where $\text{y} : \text{x}$ denotes a singleton type scenario such that y must-alias with x , namely $\text{y} = \text{x}$. The last two cases preserve separation types, whose types can be mutated. There may be some other possibilities, such as a separation type and a pure type. For simplicity, we omit such mixed scenarios. We also omit considerations for the uniqueness type, since separation types are strictly more expressive.

Lastly, we reiterate that separation types are designed to manage ownership and permission to access disjoint regions of memory. In particular, separation types support must aliasing, while pure types allow arbitrary aliasing. We thus design each separation type $\text{x} \mapsto \text{T}$ to be mutually disjoint from pure type $\text{x} : \text{T}$, as captured by the contradiction, where $\Leftrightarrow$ denotes logical equivalence.

$$x \mapsto T \wedge x : T \Leftrightarrow \text{false}$$

2.3 A New Type Hierarchy with Comprehensive Errors

Traditional type systems do not consider tracking errors via types. This seems reasonable as the type systems for statically typed languages are supposed to ensure that no type errors can occur at runtime [34]. Although programmers' wish is to eliminate *all* errors, some runtime errors are hard to eliminate completely.

Our proposal. We do not exclude errors (including occurrences of exceptions) from our reasoning. Errors are just special types in our interpretation. We thus design the type hierarchy in Fig.1 to provide a more comprehensive handling of errors. In our lattice of type hierarchy, any two types connected by some directed edge form a subtyping relation where the lower type is a subtype of its upper type.

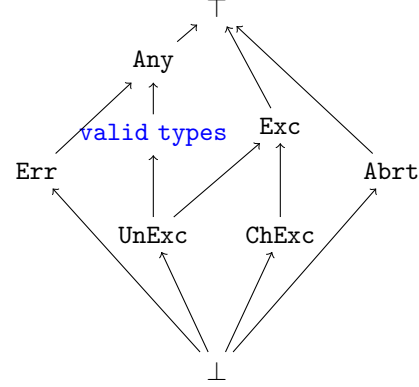


Fig. 1: Lattice of Types

Our framework supports three distinct categories of error types:

- **Abrt** type denotes that a compile-time error has occurred. Possible occurrence of **Abrt** is denoted by $\top$. Both **Abrt** and $\top$ must be absent in safe specifications.
- **Err** type, which represents a runtime error and is disjoint from all primitive and user-defined types, such as **Str**, **Int** and **List(Int)**.
- **Exc** type, which can be managed by exception-handling. It consists of two subtypes: **ChExc** for checked exceptions and **UnExc** for unchecked exceptions. Note that while **Exception** itself is a type in Java, its raised occurrences are not considered as types. In our proposal, $r:\text{Exc}$ is a pure type that signifies an occurrence of a raised exception when the value of r is being computed.

The **Abrt** and $\top$ type must be proven to be *unreachable* by our type-safe verification framework. Our *refined* motto is that “*well-typed programs must never abort*” rather than “*well-typed programs cannot go wrong*”. Consider `map`:

```
map(f, xs) = match xs of { Nil -> Nil;
                        Cons(y, ys) -> Cons(f(y), map(f, ys)) }
```

We can specify its type using the type specification with `case spec`:

```
map: ∀A, B. case[f, xs] { f:Any ∧ xs:Nil ⇒ ens[r] r:Nil;
                       f:A→B ∧ xs:Cons(A, List(A)) ⇒ ens[r] r:Cons(B, List(B));
                       _ ⇒ ens[r] r:⊥ }
```

The first two cases (with pre/post) captured the valid type specifications, while the otherwise clause triggers a compile-time error if inputs f and xs are not of the required type expected from the first two cases. Explicitly specifying

$\top$ or **Abrrt** can be used to flag occurrences of compile-time type errors, but are not strictly required. Whenever our system cannot find a valid type specification for some reachable method call(s), the program is marked as not well-typed.

Both **Err** and valid types are sub-types of **Any**, while **UnExc** is considered a subtype of all valid types. This type hierarchy is organized in such a way that **Err**, **ChExc** and *valid types* must be explicitly tracked in our type specifications. The set of *valid types* include both *separation* types ($<:\text{Any}_s$) and *pure* types ($<:\text{Any}_p$) that can either be primitive types or user-defined types. We provide four basic values, **err**:**Err**, **chEx**:**ChExc**, **unEx**:**UnExc** and **abort**:**Abrrt** corresponding to each of the four major error types. Additionally, **unDef**:**Err** can be used to track uninitialized value errors, while **null**:**Err** can be used to track null pointer dereferencing errors. These runtime error values are tolerated in programs and do not trigger compile-time errors, until they are actually used/accessed.

In our type hierarchy, $\top$ denotes the whole universe of values including exceptions and aborts, while **Any** denotes the set of data types that can be passed as arguments of user-defined methods. Unlike prior work [7], where $\perp$ denotes empty set since non-termination ∞ was not considered a value, our semantic meaning for $\perp$ in this paper is a singleton set $\{\infty\}$, where ∞ is also present in all types. An example of $\perp$ is the following function **loop**:

```
(* Type specification/code of loop *)
loop : req[x] x:Any ens[r] r: $\perp$ 
loop(x) = loop(x)
```

Our Hoare logic framework can be used to prove the above holds, and thus proves the presence of non-termination for **loop**(**x**). Here **x**: $\perp$ is equivalent to **x**: ∞ and is a strong condition, denoting falsehood, since computing a value for ∞ leads to unreachability. In contrast, its complement, $\neg\perp$ denoting $\top - \{\infty\}$, would denote termination. Ascertaining $\neg\perp$ type precisely requires termination proofs to be shown for the codes of such programs. Here again, we can draw on past research from the verification community [22, 23] on proving terminating codes by the use of ranking measures. For simplicity, we shall omit the consideration of terminating types and use the over-approximation (or weakening) $\neg\perp \subset \top$.

3 Core Language and a Logic for Types

We consider a strict higher-order functional language with mostly immutable data constructors with the exception of **Ref** type. This mutable **Ref** type uses type specifications outlined earlier in Sec 2.2. This section will focus on key features needed to support both static and dynamic typings. To support the latter, we will use a lattice of types, where data constructors and primitive types are organized into a subtype hierarchy with **Any** as the supertype of all valid types, covering both user-defined and primitive types. Each **let** binding introduces an immutable variable. Functions and data constructors are always fully applied. Function definitions and lambda abstractions are always given suitable type specifications. We also provide an alternate style for function specifications,

$\lambda x^* r. \Phi[r]$, so that **case** and **ens** constructs do not need to explicitly track the parameters/result. For simplicity, we omit **inout** parameters.

(Prog) $prog ::= def_1 ; \dots ; def_n$
 (Defn) $def ::= f \ x_1 \dots x_n = e \text{ where } f :: \lambda x_1 \dots x_n r. \Phi[r]$
 (Full) $e ::= val \mid \text{let } x:t = e_1 \text{ in } e_2 \mid C(e_1 \dots e_n) \mid (f \ e_1 \dots e_n) \mid (t) \ e$
 $\mid \text{match } e \{ p_1 \rightarrow e_1 \mid \dots \mid p_n \rightarrow e_n \}$
 (Value) $val ::= x \mid c \mid \text{err} \mid \text{abrt} \mid \lambda x^*. e :: \lambda x^* r. \Phi[r]$
 (Core) $e ::= val \mid \text{let } x = e_1 \text{ in } e_2 \mid C(x_1 \dots x_n) \mid (f \ x_1 \dots x_n) \mid (t) \ x$
 $\mid \text{match } x \{ p_1 \rightarrow e_1 \mid \dots \mid p_n \rightarrow e_n \}$
 (Base) $base ::= c \mid x \mid Bool \mid Int \mid Str \mid Ref(\dots) \mid \dots \mid Err \mid Abrt \mid Any \mid \perp \mid \top$
 $\mid C(t_1 \dots t_n) \mid pred(t_1 \dots t_n)$
 (Type) $t ::= base \mid t_1 \rightarrow t_2 \mid t_1 \wedge t_2 \mid t_1 \vee t_2 \mid \neg t$
 (Pattern) $p ::= c \mid Bool \mid Int \mid Str \mid \dots \mid Err \mid Abrt \mid Any \mid C(x_1 \dots x_n) \mid pred(-)$
 $\mid \rightarrow - \mid p_1 \wedge p_2 \mid p_1 \vee p_2 \mid \neg p \mid -$
 $c \in \mathbb{B} \cup \mathbb{Z} \cup \dots \quad x, f, r \in vars$

Fig. 2: Syntax of Full and Core Language with Types

We also provide a type cast operator $(t) \ x$ to ensure that we are computing values of the correct type. For statically-typed programs, we expect all cast operations to be proven safe at compile-time. For gradually-typed programs, some residual cast operations may remain for run-time checking. However, the mechanisms for gradual-typing are omitted in the present paper. We apply preprocessing from *(Full)* to *(Core)* via equivalence transformation $\Leftrightarrow$. When translating strict arguments, we apply type-casting to **Any** to trigger and propagate **Abrt** errors. This is currently done for data constructors and function calls, and match constructs. We also translate **let** construct to make explicit its propagation of **Abrt**. This allows a simpler **let** construct in our core language.

$$\begin{aligned}
 C(e_1, \dots, e_n) &\Leftrightarrow \text{let } \{x_i: \text{Any} = e_i\}_{i=1}^n \text{ in } C(x_1, \dots, x_n) \\
 f \ e_1 \dots e_n &\Leftrightarrow \text{let } \{x_i: \text{Any} = e_i\}_{i=1}^n \text{ in } f \ x_1 \dots x_n \\
 \text{match } e \{ \dots \} &\Leftrightarrow \text{let } x: \text{Any} = e \text{ in match } x \{ \dots \} \\
 \text{let } x:t = e_1 \text{ in } e_2 &\Leftrightarrow \text{let } x = (t) \ e_1 \text{ in match } x \{ \text{Abrt} \rightarrow x \mid _ \rightarrow e_2 \} \\
 (t) \ e &\Leftrightarrow \text{let } x = e \text{ in } (\text{let } x_1 = (t) \ x \text{ in } x_1)
 \end{aligned}$$

The type and pattern (used in match construct) are defined as in Fig. 2. Take note that $\top$ and $\perp$ are not included in match patterns, since the former trivially holds, while testing for the latter is equivalent to the halting (or non-termination) problem. Note that $C(x_1 \dots x_n)$ uses immutable fields.

For ease of programming, we allow the patterns in each match construct to overlap, and can rely on top-down processing to give an unambiguous semantics. As an example, consider a construct **match** $x \{ p_1 \rightarrow e_1 \mid p_2 \rightarrow e_2 \mid p_3 \rightarrow e_3 \}$ with three branches where p_1, p_2 and p_3 may overlap. We must ensure that the patterns in each match construct never overlap, by translating (before Hoare-style reasoning) to **match** $x \{ p_1 \rightarrow e_1 \mid p_2 \wedge \neg(p_1) \rightarrow e_2 \mid p_3 \wedge \neg(p_1 \vee p_2) \rightarrow e_3 \}$.

Hoare rules can deal with complex patterns formed from $\wedge$, $\vee$, and $\neg$. To support efficient runtime execution, we would need to translate each match construct to use only simple patterns. This translation is meant to support efficient

runtime execution and is not required for Hoare-style reasoning. The rules to translate complex type patterns to simpler patterns are summarised below.

$$\begin{aligned} \text{match } x \{ \dots \mid p_1 \vee p_2 \rightarrow e \mid \dots \} &\Leftrightarrow \text{match } x \{ \dots \mid p_1 \rightarrow e \mid p_2 \rightarrow e \mid \dots \} \\ \text{match } x \{ \dots \mid p_1 \wedge p_2 \rightarrow e \mid \dots \} &\Leftrightarrow \text{match } x \{ \dots \mid p_1 \rightarrow \text{match } x \{ p_2 \rightarrow e \mid \dots \} \mid \dots \} \\ \text{match } x \{ \dots \mid \neg p \rightarrow e \mid \dots \} &\Leftrightarrow \text{match } x \{ \dots \mid p \rightarrow \text{match } x \{ \dots \} \mid \neg p \rightarrow e \} \end{aligned}$$

Type specifications are captured, as shown in Fig 3, each case construct's input preconditions are pairwise disjoint, namely $\forall i, j, i \neq j \implies \Delta_i \wedge \Delta_j = \text{false}$. We use $\text{req } \Delta_1 \text{ ens}[r] \Delta_2$ as a shorthand for $\text{case } \{ \Delta_1 \rightarrow \text{ens}[r] \Delta_2 \}$. For completeness, we always add $(_ \rightarrow \text{ens}[r] r; \top)$ for possibly reachable **Abtrt** to cover the remaining case not captured by the clauses of each case specification.

$$\begin{aligned} (\text{State}) \Delta &::= \bigvee_{i=1}^n \exists v^*. \sigma_i \wedge \pi_i \\ (\text{Heap}) \sigma &::= \text{emp} \mid x \mapsto t \mid \sigma_1 * \sigma_2 \\ (\text{Pure}) \pi &::= x:t \mid x_1 = x_2 \mid \pi_1 \wedge \pi_2 \mid \pi_1 \vee \pi_2 \mid \neg \pi \mid \exists v^*. \pi \\ (\text{Spec}) \Phi &::= \text{ens}[r] \Delta \mid \text{case } \{ \Delta_1 \Rightarrow \Phi_1; \dots; \Delta_n \Rightarrow \Phi_n \} \mid \forall x^*. \Phi \end{aligned}$$

Fig. 3: Type Logic Specification

4 Hoare Logic Rules for Type-Safety

We present a forward-style Hoare rule of the form $\{ \Delta_{\text{pre}} \} e \{ \text{ens}[r] \Delta_{\text{post}} \}$, in Fig 4, which when given an input type pre-state Δ_{pre} is able to compute the strongest post-state, as Δ_{post} , that results from the evaluation of the expression e . This expression's result (and its type) is bound to r . We have two structural rules, **Conseq** and **Frame**, courtesy of Hoare logic with separation types. (Note that $\vdash$ denotes intuitionistic implication.) While we support higher-order functions in our Hoare rules, we currently restrict function type parameters to use flow-insensitive pure types, since we are relying on a two-stage pre/post type specification, and without the use of higher-order constraints. This essentially allows such functions to be treated as pure functions without any type mutation.

Simple parameter variables are earmarked to allow singleton types (see Appendix A.1), which helps reduce the use of quantified type variables. Our Hoare rules for function calls rely on type specification being provided for each of the methods, using $f::\lambda x^*. r. \Phi$. The type-specification must be in case-spec form, that would be merged into its pre-state Δ , using $(\Delta; \Phi)$. This merger is systematically carried out using the following two reduction rules.

$$\begin{aligned} \Delta; \text{case } \{ \Delta_1 \Rightarrow \Phi_1; \dots; \Delta_n \Rightarrow \Phi_n \} &\Rightarrow ((\Delta \vdash \Delta_1); \Phi_1) \vee \dots \vee ((\Delta \vdash \Delta_n); \Phi_n) \\ \Delta; \text{ens}[r] \Delta_2 &\Rightarrow \Delta * \Delta_2 \end{aligned}$$

We introduce a special conjunction $\vdash \wedge$ that can perform a case analysis on both separation and pure types. With pure types, this operator is equivalent to:

$$(\Delta \vdash \wedge \pi) \Leftrightarrow (\Delta \wedge \pi)$$

As an example, consider a call $(f \ x)$ where $f : \text{req } [x] x:\text{Int} \text{ ens}[r] r:\text{Int}$. Its expanded case specification $f : \Phi$ would have been:

$$\begin{array}{c}
 \frac{\Delta_1 \vdash \Delta_3 \quad \{\Delta_3\} e \{ \text{ens}[r] \Delta_4 \} \quad \Delta_4 \vdash \Delta_2}{\{\Delta_1\} e \{ \text{ens}[r] \Delta_2 \}} \text{Conseq} \quad \frac{\{\Delta_1\} e \{ \text{ens}[r] \Delta_2 \}}{\{\Delta_1 * \Delta\} e \{ \text{ens}[r] \Delta_2 * \Delta \}} \text{Frame} \\
 \\
 \frac{\text{fresh } r}{\{\Delta\} x \{ \text{ens}[r] \Delta \wedge r : x \}} \text{Var} \quad \frac{\text{fresh } r, T_1..T_n \quad \Delta_1 \equiv (\Delta \wedge \bigwedge_{i=1}^n x_i : T_i)}{\{\Delta\} C(x_1..x_n) \{ \text{ens}[r] \Delta_1 \wedge r \mapsto C(T_1..T_n) \}} \text{Constr} \\
 \\
 \frac{\text{fresh } r \quad v ::= c \mid \text{err} \mid \text{abrt}}{\{\Delta\} v \{ \text{ens}[r] \Delta \wedge r : \text{ty}(v) \}} \text{Val} \quad \frac{\text{fresh } r \quad \Delta \vdash f :: \lambda x^* r. \Phi}{\{\Delta\} f(x^*) \{ \text{ens}[r] \Delta; \Phi \}} \text{Call} \\
 \\
 \frac{\text{fresh } r \quad \Delta_1 \equiv (\Delta \wedge x : t \wedge r : x) \vee (\Delta \wedge x : \neg t \wedge r : \text{Abrt})}{\{\Delta\} (t) x \{ \text{ens}[r] \Delta_1 \}} \text{Cast} \\
 \\
 \frac{\{\Delta\} e_1 \{ \text{ens}[r_1] \Delta_1 \} \quad \{[r_1 := x] \Delta_1\} e_2 \{ \text{ens}[r] \Delta_2 \}}{\{\Delta\} \text{let } x = e_1 \text{ in } e_2 \{ \text{ens}[r] \Delta_2 \}} \text{Let} \\
 \\
 \frac{\text{fresh } r \quad \{\Delta \wedge (x : p_1 \vee x \mapsto p_1)\} e_1 \{ \text{ens}[r] \Delta_1 \} \cdots \{\Delta \wedge (x : p_n \vee x \mapsto p_n)\} e_n \{ \text{ens}[r] \Delta_n \}}{\{\Delta\} \text{match } x \{ p_1 \rightarrow e_1 \mid \cdots \} \{ \text{ens}[r] \bigvee_{i=1}^n \Delta_i \}} \text{Match} \\
 \\
 \frac{\text{fresh } f \quad v^* = \text{vars}(\Delta) - \text{vars}(e) \cup \{x^*\} \quad \{\exists v^*. \text{Pure}(\Delta)\} e \{ \downarrow \Phi \}}{\{\Delta\} \lambda x^*. e :: \lambda x^* r. \Phi \{ \text{ens}[f] \Delta \wedge f :: \lambda x^* r. \Phi \}} \text{Lambda}
 \end{array}$$

Fig. 4: Forward Hoare Rules for Type Safety

$$\Phi \equiv \text{case}[x] \{ x : \text{Int} \Rightarrow \text{ens}[r] r : \text{Int}; x : \neg \text{Int} \Rightarrow \text{ens}[r] r : \text{Abrt} \}$$

Given a program state $\Delta_{\text{pre}} \equiv x : \text{Int}$, the composition $\Delta_{\text{pre}}; \Phi$ would result in $x : \text{Int} \wedge r : \text{Int}$. However, if the initial program state was $\Delta_{\text{pre}} \equiv x : \text{Any}$, our merged composition would result in $x : \text{Int} \wedge r : \text{Int} \vee x : (\text{Any} \wedge \neg \text{Int}) \wedge r : \text{Abrt}$ instead, providing us with a case-analysis outcome for $\Delta_{\text{pre}}; \Phi$.

In the presence of separation type σ , the $\vdash \wedge$ operator need to perform a bi-abductive heap-entailment to obtain residual frame Δ_s , as shown below.

$$\frac{\Delta \wedge \pi_0 \vdash \sigma \rightsquigarrow \Delta_s}{\Delta \vdash \wedge \sigma \wedge \pi \Leftrightarrow \Delta_s \wedge \pi \wedge \pi_0}$$

The **Cast** rule results in two states, depending on whether the cast to type (t) succeeds or not. The **Let** rule propagates the type state we obtained from e_1 to e_2 . **Abrt** failure is propagated by the pre-processing transformation of Sec 3. The **Match** rule results in an exhaustive post-state in disjunctive form. Lastly, when processing lambda abstractions, we use another Hoare rule $\{\Delta\} e \{ \downarrow \Phi \}$ in Fig 5 for specification checking, where Φ is supplied as an input to help build suitable pre-states for checking the method body e satisfies type specification Φ .

$$\begin{array}{c}
 \frac{\{\Delta * \Delta_1\} e \{ \downarrow \Phi_1 \} \cdots \{\Delta * \Delta_n\} e \{ \downarrow \Phi_n \}}{\{\Delta\} e \{ \downarrow \text{case } \{\Delta_1 \Rightarrow \Phi_1; \cdots; \Delta_n \Rightarrow \Phi_n\} \}} \text{Spec-Case} \\
 \\
 \frac{\{\Delta\} e \{ \text{ens}[r] \Delta_1 \} \quad \Delta_1 \vdash \Delta_2}{\{\Delta\} e \{ \downarrow \text{ens}[r] \Delta_2 \}} \text{Spec-Ens}
 \end{array}$$

Fig. 5: Checking Hoare Rules for Type Specification

5 Soundness of Our Type Specification Framework

We have formalized our Hoare Logic with pure and separation types to ensure type safety in Rocq and proved its soundness [1]. We define the semantics of types using a logical relation (Fig. 6). Different from the pen-and-paper proof, we do not rely on explicit stores and manipulation of names, instead representing variables using Rocq variables. As such, and in keeping with the use of semantic subtyping, types are shallowly embedded as functions of type $\text{val} \rightarrow \text{Prop}$. Proving a type assertion $v:t$ thus amounts to proving the proposition $\mathbf{t} \ v$ by semantic reasoning. Logical connectives for building types in Fig. 6 are lifted directly from their propositional counterparts. Subtyping is defined similarly, as a lifting of implication to type propositions: $t_1 <: t_2 \equiv \forall v, v:t_1 \implies v:t_2$. This encoding is expressive, supporting even the dependent singleton types of Appendix A.1. Due to the lack of state and non-(co)inductive types, step-indexing is not required. (Co)inductive types are represented using the (co)inductive types of Coq.

$\mathbf{s}, \mathbf{h} \models \mathbf{x}:\mathbf{c}$	<i>iff</i>	$\mathbf{s}(\mathbf{x})=\mathbf{c}$	$\mathbf{s}, \mathbf{h} \models \mathbf{x}_1:\mathbf{x}_2$	<i>iff</i>	$\mathbf{s}(\mathbf{x}_1)=\mathbf{s}(\mathbf{x}_2)$
$\mathbf{s}, \mathbf{h} \models \mathbf{x}_1=\mathbf{x}_2$	<i>iff</i>	$\exists l \cdot \mathbf{s}(\mathbf{x}_1)=\mathbf{s}(\mathbf{x}_2)=l$			
$\mathbf{s}, \mathbf{h} \models \Delta_1 \vee \Delta_2$	<i>iff</i>	$\mathbf{s}, \mathbf{h} \models \Delta_1$ or $\mathbf{s}, \mathbf{h} \models \Delta_2$			
$\mathbf{s}, \mathbf{h} \models \sigma \wedge \pi$	<i>iff</i>	$\exists \mathbf{h}_1, \mathbf{h}_2 \cdot \mathbf{h} = \mathbf{h}_1 \mathbf{o} \mathbf{h}_2 \wedge \mathbf{s}, \mathbf{h}_1 \models \sigma \wedge \mathbf{s}, \mathbf{h}_2 \models \pi$			
$\mathbf{s}, \mathbf{h} \models \sigma_1 * \sigma_2$	<i>iff</i>	$\exists \mathbf{h}_1, \mathbf{h}_2 \cdot \mathbf{h} = \mathbf{h}_1 \mathbf{o} \mathbf{h}_2 \wedge \mathbf{s}, \mathbf{h}_1 \models \sigma_1 \wedge \mathbf{s}, \mathbf{h}_2 \models \sigma_2$			
$\mathbf{s}, \mathbf{h} \models \pi_1 \vee \pi_2$	<i>iff</i>	$\mathbf{s}, \mathbf{h} \models \pi_1$ or $\mathbf{s}, \mathbf{h} \models \pi_2$			
$\mathbf{s}, \mathbf{h} \models \pi_1 \wedge \pi_2$	<i>iff</i>	$\mathbf{s}, \mathbf{h} \models \pi_1$ and $\mathbf{s}, \mathbf{h} \models \pi_2$			
$\mathbf{s}, \mathbf{h} \models \neg \pi$	<i>iff</i>	not $(\mathbf{s}, \mathbf{h} \models \pi)$			
$\mathbf{s}, \mathbf{h} \models \Delta_1 * \Delta_2$	<i>iff</i>	$\mathbf{s}, \mathbf{h} \models (\sigma_1 * \sigma_2) \wedge (\pi_1 \wedge \pi_2)$ where $\Delta_i = \sigma_i \wedge \pi_i$			
$\mathbf{s}, \mathbf{h} \models \mathbf{x}:\mathbf{t}$	<i>iff</i>	$\exists l \cdot \mathbf{s}(\mathbf{x})=l$ and $\mathbf{s}, \mathbf{h} \models l:\mathbf{t}$			
$\mathbf{s}, \mathbf{h} \models \mathbf{l}:\text{prim}$	<i>iff</i>	$\mathbf{l} \in \text{prim}$ where $\text{prim} = \text{Int} \mid \text{Str} \mid \text{Err} \mid \dots$			
$\mathbf{s}, \mathbf{h} \models \mathbf{l} \mapsto \mathbf{C}(\mathbf{t}_1, \dots, \mathbf{t}_n)$	<i>iff</i>	$\mathbf{h} = \{\mathbf{l} \mapsto \mathbf{C}(\mathbf{l}_1, \dots, \mathbf{l}_n)\}$ and $\bigwedge_{i=1}^n \mathbf{s}, \mathbf{h} \models \mathbf{l}_i:\mathbf{t}_i$			
$\mathbf{s}, \mathbf{h} \models \mathbf{l}:\mathbf{C}(\mathbf{t}_1, \dots, \mathbf{t}_n)$	<i>iff</i>	$\mathbf{h}(\mathbf{l}) = \mathbf{C}(\mathbf{l}_1, \dots, \mathbf{l}_n)$ and $\bigwedge_{i=1}^n \mathbf{s}, \mathbf{h} \models \mathbf{l}_i:\mathbf{t}_i$			
$\mathbf{s}, \mathbf{h} \models \mathbf{l}:\text{pred}(\mathbf{t}^*)$	<i>iff</i>	$\mathbf{l}:\text{pred}(\mathbf{t}^*) = \Delta$ and $\mathbf{s}, \mathbf{h} \models \Delta$			
$\mathbf{s}, \mathbf{h} \models \mathbf{l}:\neg \mathbf{t}$	<i>iff</i>	not $(\mathbf{s}, \mathbf{h} \models \mathbf{l}:\mathbf{t})$			
$\mathbf{s}, \mathbf{h} \models \mathbf{l}:\mathbf{t}_1 \rightarrow \mathbf{t}_2$	<i>iff</i>	$\forall v \cdot (\mathbf{s}, \mathbf{h} \models v:\mathbf{t}_1 \rightarrow \mathbf{s}, \mathbf{h} \models \mathbf{l}(v):\mathbf{t}_2)$			

Fig. 6: Semantics of Type Logic Formulae

The case for functions agrees with the definition of the Hoare triple. This enables the smooth refinement of typed specifications into logical ones.

$$\{\Delta_1\} e \{ \text{ens}[r] \Delta_2 \} \quad \text{iff} \quad (\Delta_1 \implies \forall r, e \rightsquigarrow r \implies r \models \Delta_2) \quad (1)$$

$$f:\mathbf{t}_1 \rightarrow' \mathbf{t}_2 \quad \text{iff} \quad \forall v, \{v:\mathbf{t}_1\} f(v) \{ \text{ens}[r] r:\mathbf{t}_2 \} \quad (2)$$

Defining a modified function arrow $\rightarrow'$ in terms of a standard Hoare triple (which is also defined extensionally, using a big-step relation), we can see that it is equivalent (a lifting of $\iff$) to the regular function arrow from Fig. 6.

Lemma 1 (Agreement between function arrows). $t_1 \rightarrow t_2 \equiv t_1 \rightarrow' t_2$.

In other words, the behaviors of well-typed programs are exactly those of programs specified by Hoare triples.

Next, we prove the soundness of the rules used in Fig. 4. The statement of the soundness theorem is essentially the definition of the validity of Hoare triples.

Theorem 1 (Soundness). *Given $\{\Delta_1\} e \{\text{ens}[r] \Delta_2\}$, $\forall s, h, s', h', v$ if $s, h \models \Delta_1$ holds and $s, h, e \rightsquigarrow s', h', v$, then $s' + [(r, v)], h' \models \Delta_2$.*

Proof. The proof relies on compatibility lemmas over constructs in Fig. 4.

Definition 1 (Well-typed programs never abort). *e is well-typed under satisfiable Δ_1 if and only if $\{\Delta_1\} e \{\text{ens}[r] \Delta_2\}$ and Δ_2 does not contain **Abt**.*

We use the term “well-typed” with respect to the pre/post specifications. Hence, programs are well-typed based on the scenarios that the specifications describe. We also use big-step semantics in Theorem 1. Big-step semantics will ensure that either an expression is non-terminating, or if it terminates it will eventually reach its post-condition. Hence, Theorem 5.2 (using big-step semantics) guarantees both type preservation (for pure types) and progress. In contrast, traditional type systems typically rely on small-step semantics, which would then require two theorems, one for type preservation and another to show progress.

Case study	Lemmas	LoC	Rec	HO	Sub	Sing
id (Appendix A.1)	2	29	✗	✗	✗	✓
tail (Section 1)	5	110	✗	✗	✓	✗
apply (Section 1)	3	113	✗	✓	✗	✓
length (Appendix A.2)	1	49	✓	✗	✗	✗

Table 1: Case studies

We conclude with an overview of the case studies presented in Table 1, which are simple examples used in the paper. The LoC column counts both lines of definition and proof, and serves as an approximation of the difficulty of each case study. The four columns after it indicate what features each case study has: recursion (Rec), higher-order (HO), subtyping (Sub), or singleton types (Sing).

All definitions and case studies are contained in our development, which is available together with the paper. The only axiom used pervasively is the law of excluded middle, which is required in proofs of type safety concerning negated types. The development contains 66 lemmas over 1153 LoC and builds on the formalization for the *Separation Logic Foundations* book [8]. As [8] currently supports only *flow-insensitive* pure types, we have to expand its library to include support for our newly proposed *flow-sensitive* separation types.

6 Implementation and Evaluation.

We prototype our type specification framework into Heifer-type¹, a verifier implemented using OCaml on top of Heifer [15], using forward compositional analysis. Specifically, given a program written in our core language (shown in Fig. 2) with annotated type specifications, Heifer-type verifies if the implementations satisfy the specifications. Heifer-type implements the proof rules in Fig. 5 and

¹ Heifer-type is publicly available at <https://github.com/hipsleek/Heifer-type>

a proof search algorithm. The algorithm contains a *specification map* T , that maps methods to their specifications and uses a function $post(\Delta_p, T, e)$, that computes the post-states Δ'_q for a method body e from the pre-condition Δ_p via applying the proof rules. Heifer-type also checks the validity of the specification by invoking an internal solver to check whether $\Delta'_q \models \Delta_q * \text{emp}$ where Δ_q is the given post-condition. This solver is a lightweight entailment checker that is not SMT-based. It only performs heap matching and subtyping checking which are enough for solving the constraints allowed in our type formulas. If the checking is successful, this specification will be marked as verified in T .

Benchmark	Heifer-type			OCaml TP	Benchmark	Heifer-type			OCaml TP
	LoC	LoS	T			LoC	LoS	T	
basic	30	13	< 0.01	✓	map	2	3	< 0.01	✓
stateful_funs	4	3	< 0.01	✗	tail	2	2	< 0.01	-
refs_pure	6	4	< 0.01	✓	length	3	2	< 0.01	-
refs_sep	6	5	< 0.01	✗	length_generic	4	3	< 0.01	✗
swap_pure	4	1	< 0.01	✓	shape_funs	6	2	< 0.01	✗
swap_sep	5	3	< 0.01	✗					

Table 2: A Comparison with OCaml type system.

We have conducted some initial evaluations for our implementation. Table 2 shows the results for the set of benchmarks we have been using for the comparison between our system and OCaml type system. The table records: 1) LoC, the number of lines of code; 2) LoS, the number of lines of specifications - one pair of pre/post per line; 3) the running time (in seconds). For the OCaml type system, ✓ indicates that OCaml type system can type that program and have the equivalent type annotation as ours; ✗ indicates that the program is not typeable in OCaml type system and lastly, - means that OCaml type system can type that program but is less precise. We have discussed the most important benchmarks in the main content of the paper and demonstrated why our system could be more accurate in handling path-sensitivity, flow-sensitivity and errors. As our benchmarks are considered small and the specification language is lightweight, the verification time is also very fast (less than 0.01 seconds).

7 Related Works

Semantic Subtyping. Semantic-based subtyping interprets each type in terms of a set of values. In general, a type t is viewed as the set of values having that type. Let $\text{Set}(t)$ be the set of all values of type t . Then, the subtyping relation can be expressed via set inclusion, i.e., $t_1 <: t_2 \Leftrightarrow \text{Set}(t_1) \subseteq \text{Set}(t_2)$. The advantage of semantic subtyping has been explored in [6, 16, 5]. Later, several works extended semantic subtyping from different perspectives: on the subtyping relation computation [4]; on the decidability [18]; on its use in other programming paradigms [2, 26]. Singleton, union, and intersection types have gained more attention through their set-based interpretation of types [12, 11, 9]. Using a syntactic approach via Boolean algebra of types, MLstruct [25] infers principal type with union and intersection for an ML-like language. MLstruct takes subtyping into account, while subtyping is avoided in traditional Hindley-Milner type

inference. Later, intersection and union type inferencing with semantics-based subtyping was also extended to dynamically-typed languages [7].

Hoare Logic for Type Safety. Hoare logic has also been used to ensure type safety for selected domains, such as typed assembly (used in operating systems) [33, 20], for dynamically-typed programs in [14, 13]. They extract type information from the precondition and convert it to a type assertion p . Type safety is ensured if $\{p\}e\{true\}$ is verified. Our work extensively follows this approach.

Type Systems for Memory and Resource Safety. Sub-structural type systems have also been developed to reason about system resources, such as memory, files, and locks [27]. One of the most notable examples is the use of ownership and borrowing mechanisms in the Rust language to ensure memory safety. Recently, OCaml, a variant of OCaml, has also been extended with a linear type system to reason about affinity, uniqueness, and locality of resources, which can be used to ensure memory safety and also data race freedom [17, 24]. While we share a similar goal with these works, our approach is different. We leverage the expressivity and flexibility of Hoare logic to implement the type verification system and reason about memory and resource properties through the introduction of separation types and flow sensitivity.

From Uniqueness to Separation Type. We are certainly not the first to use resource and ownership types to support strong aliasing and flow-sensitivity in type systems. A recent work [3] related frame rules used in the uniqueness type system and separation logic via an FFI. In the proposal for typed assembly, [32, 28] showed how to describe various shapely data structure shapes via alias types. Subsequently, ownership type was used to support strong updates for refinement type system by [31]. Some proposals even used refinement separation types to support safer asynchronous execution [21]. By leveraging Hoare logic with separation types, we have likewise increased our coverage on type-safety.

8 Conclusion

This paper has introduced an enhanced logic of types that is capable of ensuring stronger type safety for both statically-typed and dynamically-typed programming languages. We have built upon the works from semantic-based sub-typing, and developed it into a more expressive logic of types by adding a novel *separation type* with *strong update* capability. We have also proposed a comprehensive error-handling scheme that include **Err** for runtime errors that must be tracked explicitly, **Exc** for unchecked and checked exceptions, and **Abt** for serious errors that are flagged by our type system. Our framework provides *path-sensitive* and *flow-sensitive* type specifications that support type mutations. We have also designed a small core language that includes a versatile **match** construct that can be used for both *type-testing*, *pattern-matching* and also *exception handling*. Lastly, separation type is extremely expressive and can also be used to support data structure invariance and data-race freedom. App D.3 and App D.5 highlight how we intend to support these extensions, as part of our future work.

Bibliography

- [1] Rocq proof for soundness. <https://github.com/hipsleek/stagedtypes>.
- [2] Davide Ancona and Andrea Corradi. Semantic subtyping for imperative object-oriented languages. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2016*, pages 568–587. ACM, 2016.
- [3] Pilar Selene Linares Arévalo, , Arthur Azevedo de Amorim, Vincent Jackson, Liam O’Connor, Peter Schachte, and hristineRizkallah. Memory safety: Uniqueness as separation. In *Programming Languages and Systems - 23rd Asian Symposium, APLAS 2025*, volume 16201 of *Lecture Notes in Computer Science*, pages 3–21. Springer, 2025.
- [4] Gavin M. Bierman, Andrew D. Gordon, Catalin Hritcu, and David E. Longworthy. Semantic subtyping with an SMT solver. In *Proceeding of the 15th ACM SIGPLAN international conference on Functional programming, ICFP 2010*, pages 105–116. ACM, 2010.
- [5] Giuseppe Castagna. Semantic subtyping: Challenges, perspectives, and open problems. In *Theoretical Computer Science, 9th Italian Conference, ICTCS 2005*, volume 3701 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2005.
- [6] Giuseppe Castagna and Alain Frisch. A gentle introduction to semantic subtyping. In *Proceedings of the 7th International ACM SIGPLAN Conference on Principles and Practice of Declarative Programming*, pages 198–199. ACM, 2005.
- [7] Giuseppe Castagna, Mickaël Laurent, and Kim Nguyen. Polymorphic type inference for dynamic languages. *Proc. ACM Program. Lang.*, 8(POPL):1179–1210, 2024.
- [8] Arthur Charguéraud. *Separation Logic Foundations*, volume 6 of *Software Foundations*. Electronic textbook, 2024. Version 2.2, <http://softwarefoundations.cis.upenn.edu>.
- [9] Bruno C. d. S. Oliveira, Zhiyuan Shi, and João Alpuim. Disjoint intersection types. In *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, ICFP 2016*, pages 364–377. ACM, 2016.
- [10] Stephen Dolan and Alan Mycroft. Polymorphism, subtyping, and type inference in MLsub. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages, POPL 2017*, pages 60–72. ACM, 2017.
- [11] Jana Dunfield. Elaborating intersection and union types. In Peter Thiemann and Robby Bruce Findler, editors, *ACM SIGPLAN International Conference on Functional Programming, ICFP’12, Copenhagen, Denmark, September 9-15, 2012*, pages 17–28. ACM, 2012.
- [12] Richard A. Eisenberg and Stephanie Weirich. Dependently typed programming with singletons. In *Proceedings of the 5th ACM SIGPLAN Symposium on Haskell, Haskell 2012*, pages 117–130. ACM, 2012.

- [13] Björn Engelman and Ernst-Rüdiger Olderog. A sound and complete Hoare logic for dynamically-typed, object-oriented programs. In *Theory and Practice of Formal Methods - Essays Dedicated to Frank de Boer on the Occasion of His 60th Birthday*, volume 9660 of *Lecture Notes in Computer Science*, pages 173–193. Springer, 2016.
- [14] Björn Engelman, Ernst-Rüdiger Olderog, and Nils Erik Flick. Closing the gap - formally verifying dynamically typed programs like statically typed ones using Hoare logic - extended version. *CoRR*, abs/1501.02699, 2015.
- [15] Darius Foo, Yahui Song, and Wei-Ngan Chin. Staged specification logic for verifying higher-order imperative programs. In *International Symposium on Formal Methods*, pages 501–518. Springer, 2024.
- [16] Alain Frisch, Giuseppe Castagna, and Véronique Benzaken. Semantic subtyping. In *Proceedings 17th Annual IEEE Symposium on Logic in Computer Science*, pages 137–146. IEEE, 2002.
- [17] Aïna Linn Georges, Benjamin Peters, Laila Elbeheiry, Leo White, Stephen Dolan, Richard A. Eisenberg, Chris Casinghino, François Pottier, and Derek Dreyer. Data race freedom à la mode. *Proc. ACM Program. Lang.*, 9(POPL):656–686, 2025.
- [18] Nils Gesbert, Pierre Genevès, and Nabil Layaïda. A logical approach to deciding semantic subtyping. *ACM Trans. Program. Lang. Syst.*, 38(1):3:1–3:31, 2015.
- [19] Cristian Gherghina, Cristina David, Shengchao Qin, and Wei-Ngan Chin. Structured specifications for better verification of heap-manipulating programs. In *FM 2011: Formal Methods - 17th International Symposium on Formal Methods*, volume 6664 of *Lecture Notes in Computer Science*, pages 386–401. Springer, 2011.
- [20] Nadeem Abdul Hamid and Zhong Shao. Interfacing Hoare logic and type systems for foundational proof-carrying code. In *Theorem Proving in Higher Order Logics, 17th International Conference, TPHOLs 2004*, volume 3223 of *Lecture Notes in Computer Science*, pages 118–135. Springer, 2004.
- [21] Johannes Kloos, Rupak Majumdar, and Viktor Vafeiadis. Asynchronous liquid separation types. In *29th European Conference on Object-Oriented Programming (ECOOP 2015)*, pages 396–420. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2015.
- [22] Ton Chanh Le, Cristian Gherghina, Aquinas Hobor, and Wei-Ngan Chin. A resource-based logic for termination and non-termination proofs. In *Formal Methods and Software Engineering - 16th International Conference on Formal Engineering Methods, ICFEM*, volume 8829 of *Lecture Notes in Computer Science*, pages 267–283. Springer, 2014.
- [23] Ton Chanh Le, Shengchao Qin, and Wei-Ngan Chin. Termination and non-termination specification inference. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 489–498. ACM, 2015.
- [24] Anton Lorenzen, Leo White, Stephen Dolan, Richard A. Eisenberg, and Sam Lindley. Oxidizing OCaml with modal memory management. *Proc. ACM Program. Lang.*, 8(ICFP):485–514, 2024.

- [25] Lionel Parreaux and Chun Yin Chau. MLstruct: principal type inference in a boolean algebra of structural types. *Proc. ACM Program. Lang.*, 6(OOPSLA2):449–478, 2022.
- [26] Tommaso Petrucciani, Giuseppe Castagna, Davide Ancona, and Elena Zucca. Semantic subtyping for non-strict languages. In *24th International Conference on Types for Proofs and Programs, TYPES 2018*, volume 130 of *LIPICs*, pages 4:1–4:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.
- [27] Benjamin C. Pierce. *Advanced Topics in Types and Programming Languages*. MIT Press, 2004.
- [28] Frederick Smith, David Walker, and J. Gregory Morrisett. Alias types. In Gert Smolka, editor, *Programming Languages and Systems, 9th European Symposium on Programming, ESOP 2000, Held as Part of the European Joint Conferences on the Theory and Practice of Software, ETAPS 2000, Berlin, Germany, March 25 - April 2, 2000, Proceedings*, volume 1782 of *Lecture Notes in Computer Science*, pages 366–381. Springer, 2000.
- [29] Jane Street. OxCaml. <https://oxcaml.org/>, 2025.
- [30] Robert E Strom and Shaula Yemini. Typestae : A programming language concept for enhancing software reliability. *IEEE Trans. on Software Engineering.*, 12(1):157–171, 1986.
- [31] John Toman, Ren Siqi, Kohei Suenaga, Atsushi Igarashi, and Naoki Kobayashi. Consort: Context-and flow-sensitive ownership refinement types for imperative programs. In *ESOP*, volume 20, pages 684–714, 2020.
- [32] David Walker and J. Gregory Morrisett. Alias types for recursive data structures. In Robert Harper, editor, *Types in Compilation, Third International Workshop, TIC 2000, Montreal, Canada, September 21, 2000, Revised Selected Papers*, volume 2071 of *Lecture Notes in Computer Science*, pages 177–206. Springer, 2000.
- [33] Jean Yang and Chris Hawblitzel. Safe to the last instruction: automated verification of a type-safe operating system. In *Proceedings of the 2010 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2010*, pages 99–110. ACM, 2010.
- [34] Eric Zhao, Raef Maroof, Anand Dukupati, Andrew Blinn, Zhiyi Pan, and Cyrus Omar. Total type error localization and recovery with holes. *Proceedings of the ACM on Programming Languages*, 8(POPL):2041–2068, 2024.

A Basic Features of Types

A.1 Wider Usage of Singleton Types

Recently, the singleton type was used in [25, 7] for constant literals of the form c , such as 3 or “hi”. Whenever a variable v is bound to a constant, say c , we make use of the type notation $v:c$ to state that variable v has singleton type c .

In the Scala language itself, a singleton object class is essentially a class with only a single instance. In this paper, we propose to denote a singleton type as a type that contains only a single value. We propose to extend the use of singleton types to include also parameter (input) variables, such as x , using the type annotation $r:x$ to denote that some result r has singleton type x . In terms of Hoare logic, this is essentially equivalent to $r=x$, since in any such output r with the singleton type x , r itself has the same value as x .

An example of its use is in the following identity function.

```
id x = x
```

Using the singleton type, we can use type $id:(x:Any \rightarrow r:x)$, with input x of allowable input type, Any , and output r to be of the singleton type x . A good thing about such a type specification is that it requires fewer universally quantified type variables. The current type annotation approach would instead provide a principal type of form $id:(\forall a.a \rightarrow a)$ with a universal type variable a . Technically, we can prove that $id:(x:Any \rightarrow r:x)$ is equivalent to $id:(\forall a.a \rightarrow a)$. However, the former notation has fewer quantified type variable(s), and is thus easier to apply (with fewer existential instantiations) during type-checking. Our solution can take full advantage of the Hoare logic approach to type checking since it is equivalent to $id::(\lambda x r. req \ true \ ens[r] \ r:x)$ with a weakest pre-condition $true$ and a (strongest) post-condition that is equivalent to $r=x$.

For another example, consider the following function $\lambda x.x \ x$. Here, the parameter x is applied to itself. With the help of the singleton type, we can provide its function type as $\forall b.(x:x \rightarrow b) \rightarrow b$. Again, this solution utilizes one fewer quantified type variable than $\forall a,b.(a \wedge a \rightarrow b) \rightarrow b$ proposed in [7]. Fewer quantified type variables typically make it easier to support existential instantiation of type variables that are required for calls of polymorphically-typed methods. Incidentally, using our solution, we can actually prove $(\lambda x.x \ x)(\lambda x.x \ x) : \perp$ and $(\lambda x.x \ x)(\lambda x.x) : (x:Any \rightarrow r:x)$.

Both these outcomes cannot be handled by [7]. Also, $(\lambda x.x \ x)(3) : Any$ is rightly rejected as untypeable by [7] and also by our Hoare logic approach to type-checking. As the singleton type leads to the most precise typing scenario, we currently limit its use to only variables and argument of function calls, where the most precise typing is always the best option. For field positions, we shall use type variables instead of singleton types, in order to search for the best compromise under our flow-insensitive type system.

A.2 Constructors and Type Predicates

In statically-typed languages, like OCaml, one typically uses algebraic data types. An example is the polymorphic list type we used in Sec 1. Using it, we can define the type annotation for the `length` method, as follows:

```
(* Type signature and implementation of length *)
length : ∀T. List(T) → Int
length(xs) = match xs of
  {Nil -> 0;
   Cons(y,ys) -> 1 + length(ys)}
```

To support better precision and expressiveness, we allow the use of various types that are more fine-grained than the declared algebraic data types in our framework. The above `List(T)` type is semantically equivalent to the recursive type predicate:

$$r : \text{List}(T) \equiv r : \text{Nil} \vee r : \text{Cons}(T, \text{List}(T))$$

Therefore, according to semantic subtyping, the constructors `Nil` and `Cons` hold the values that are subsets of `List`. We could treat `Nil` and `Cons` as types in the specifications. We can provide a more detailed specification for the `length` function than its type declaration as follows.

```
(* Type specification of length *)
length : ∀T. case[xs]
  {Nil ⇒ ens[r] r:0;
   Cons(T,List(T)) ⇒ ens[r] r:Int}
```

Note that the output type for `Nil` case is $r : 0$. In our framework, constants can also be viewed as types. Here, we call the type `0` a singleton type and $\text{Set}(0) \subset \text{Set}(\text{Int})$. With the help of those fine-grained types, we can write more informative specifications for path-sensitive code, especially when values (with distinct constructors) of the same algebraic data type can lead to completely different outcomes.

Using (recursive) type predicates, one could thus support algebraic data types for statically-typed programs in our type specification. This idea shall be further extended to support GADT (see Appendix B.2) and type-classes (see Appendix B.1).

Though recursive predicates can be used to define stronger types, they are thought to be incompatible with dynamically-typed programs that rely on runtime type testing. While complex types, such as `List(List(T))`, can be used in static type declarations, we propose to use its unnested form, e.g. `List(-)`, for runtime type testing. Specifically, `List(-)` is approximated to `Nil ∨ Cons(-, -)` and can thus be supported by runtime type testing, as elaborated next.

A.3 Combining Type Testing with Pattern-Matching

Statically typed languages make use of pattern-matching, while dynamically typed languages rely on type-testing. There are some recent proposals that

attempt to combine these two features, including the C# language itself. We achieve this integration with an enhanced `match` construct that can be used for both pattern-matching and structural type-testing.

We illustrate below two uses of our match-constructs. The first function, `length_generic`, uses type-testing to dispatch the length computation for each data type. We utilize case specification to specify its type as follows.

```
(* Type spec and code of length *)
length_generic : ∀T. case [xs]
  {xs:Int ⇒ ens[r] r:1;
   xs:Str ⇒ ens[r] r:Int;
   xs:List(T) ⇒ ens[r] r:Int}
length_generic xs = match xs of
  {Int -> 1;
   Str -> length_of_str(xs);
   List(_) -> length(xs)}
```

Recall that `List(·) ≡ Nil ∨ Cons(·, ·)` to denote a disjunction of its predicate's outermost constructors. Using case specification and the singleton type 1 for the `length_generic` method allows us to have a more precise type specification, when compared with intersection type:

```
length_generic : (∀T. List(T) → Int) ∧
  (Int → Int) ∧ (Str → Int)
```

The method `length` was already defined in Section A.2.

A.4 Support for Mutable Data Constructors

In the OCaml language, heap mutation is currently supported by records with mutable field(s). In our core language, we intend to use only data constructors whose arguments may be mutable. In order to support field mutation, we introduce an assignment statement $x_1 := x_2$ where x_1 and x_2 are variables from either `let` bindings or pattern-matching. With these features in our core language, we can now implement the following library for the mutable `Ref(T)` type:

```
mkRef(x)      = Ref(x)
!(m)          = match m of {Ref(w) -> w}
update(m,v)   = match m of {Ref(w) -> w := v}
```

Our Hoare reasoning rules will also have to be extended to handle the assignment statement, as shown below.

$$\frac{\text{fresh } T_1, T_2 \quad \Delta_1 \equiv (\Delta \wedge x_1:T_1 \wedge x_2:T_2) \quad \Delta_1 \vdash T_2 <: T_1}{\{ \Delta \} x_1 := x_2 \{ \text{ens}[r] \Delta_1 \wedge r:() \}} \text{Assign}$$

B Advanced Types

B.1 Support for Type Classes

We briefly describe how we can support a class of more sophisticated types, known as *type-classes*, that have been adopted by Haskell and Scala for better

code reuse. As an example, the `Num` type class is used in Haskell to support adhoc polymorphism for numeric operations, such as `+`, as follows:

$$+ : \forall T \cdot \text{Num}(T) \Rightarrow T \rightarrow T \rightarrow T$$

Here, a modified type with a context $\text{Num}(t) \Rightarrow$ is used to allow a generic `+` function to be supported. Using pre/post type specification, we can easily encode this method with the help of a type-class predicate $\text{Num}(t)$ that is constructed based on instance declarations of type class `Num` given, namely:

$$\text{Num}(T:*) \equiv (T=\text{Int}) \vee (T=\text{Float}) \vee \dots$$

Such a type-class predicate can subsequently be used in preconditions of type specifications, to constrain the applicability of each overloaded operation.

$$+ :: \lambda x y r \cdot \forall T \cdot \text{req}[x, y] \ x:T \wedge y:T \\ \wedge \text{Num}(T) \ \text{ens}[r] \ r:T$$

Similarly, higher-order type classes may also be supported. An example is the `Functor` class below:

```
class Functor(F:*→*) {
  map : ∀A,B · (A→B) → F(A) → F(B)
```

Our type specification for `map` can be captured, as below, while the `Functor` predicate itself is built from its respective instance declarations.

```
map :: λ f xs r · ∀ F:*→*,A,B ·
  req[f,xs] f:(A→B) ∧ xs:F(A) ∧ Functor(F)
  ens[r] r:F(B) Functor(F:*→*) ≡ (F=List) ∨ (F=Option) ∨
  ...
```

Type classes may also be organised in an inheritance hierarchy. For example, the `Monad` type class is an extension of the `Functor` type class. This property can be captured by the lemma, $\forall F:(* \rightarrow *) \cdot \text{Monad}(F) \implies \text{Functor}(F)$.

Note that we currently disallow type-class predicates, such as $\text{Num}(_)$ and $\text{Functor}(_)$ to be used for type-testing. This is because these predicates are on types (or type constructors) rather than values.

B.2 Support for GADT

A limited form of dependent type, known as Generalised Algebraic Data Type (GADT), had made its way into both Haskell and OCaml. Here, some type parameters are instantiated based on the data constructors used. An example is the following type-safe variant of abstract syntax tree (AST), where `Term(t)` has a type `t` which depended on the data constructor used to build the AST tree. In Haskell, such a GADT can be defined as follows.

```
data Term(_) where
  Var  :: Str → Term(Int)
  CInt :: Int → Term(Int)
  Add  :: (Term(Int),Term(Int))
```

```

      → Term(Int)
App  :: (Term(A→B), Term(A)) → Term(B)
Lam  :: (Str, Term(A)) → Term(Int→A)

```

An example of its use is `Lam("x", Var "x")` which would be derived to have type `Term(Int→Int)`. In our solution, we would model this GADT using the following recursive predicate, and use it to support type-safety verification.

```

r:Term(T:*) ≡
  ∃v. v=CInt(v) ∧ v:Int ∧ T=Int
  ∨ ∃id. r=Var(id) ∧ id:Str ∧ T=Int
  ∨ ∃e1,e2. r=Add(e1,e2) ∧ e1:Term(Int) ∧ e2:Term(Int) ∧ T
    =Int
  ∨ ∃f,e,A,B. r=App(f,e) ∧ f:Term(A→B) ∧ e:Term(A) ∧ T=B
  ∨ ∃s,e,A. r=Lam(s,e) ∧ s:Str ∧ e:Term(A) ∧ T=Int→A

```

As GADT predicates, such as `Term(·)`, are used for describing values, they can be used for type-testing in the same manner as simpler predicates, like `List(·)`.

B.3 Gradual Typing for Dynamic Languages

While our more expressive type specifications have allowed us to capture more dynamically-typed programs as statically-safe, there are still examples that are currently beyond the scope of our current logic of types, despite the use of semantics subtyping and separation type. An example of this is:

```
f(x,y) = if x>5 then !y else y+1
```

For this function to be statically type-safe, we will need to use the following pre/post type specification.

```

req[x,y] x:Int /\ y:Ref(T)/\Int
ens[r]   y:Ref(T)/\r:T /\ y:Int/\r:Int

```

However, `y:Ref(T)∧Int` leads to `y:⊥` that is equivalent to *false*. This will result in a pre-condition that is too strong. One way of rectifying this is to make use of gradual typing, with the help of a special set of pure types `?t`, where type casting can be performed dynamically at runtime. Thus, we allow `y:?(Ref(T)∨Int)` to be declared in our type-specifications, as follows:

```

req[x,y] x:Int /\ y:?(Ref(T)∨Int)
ens[r]   (y:Ref(T)/\r:T) /\ (y:Int /\ r:Int);

```

With the presence of `y:?(Ref(T)∨Int)`, we can allow runtime casts to be inserted for program code for `f` to ensure that runtime type checking is performed.

```

f(x,y) = if x>5 then let z = (?Ref(_)) y in !z
        else let z = (?Int) y in z+1

```

Such programs are thus gradually typed with the possibility of runtime type aborts for some of their execution runs. Runtime cast is implemented, as follows:

```
(?p) v = match v of {p -> v; _ -> abort}
```

The Hoare rule for runtime casts would ignore the **abort** command, as follows.

$$\frac{\text{fresh } r \quad \Delta \vdash y:?(_) \quad \Delta_1 \equiv (\Delta \wedge (x:p) \wedge r:x)}{\{\Delta\} (?p) x \{ \text{ens}[r] \Delta_1 \}} R\text{-Cast}$$

C Error Handling

C.1 Different Ways of Signifying Errors

To illustrate how these different error types may be used, consider the following simple **tail** function, which expects its input to be a **Cons** node.

```
(* Customizable error handling in the ..fail.. branch *)
tail xs = match xs of { Cons(_,ys) -> ys;
                      Nil -> ..fail.. }
```

If we use the ML language, the best that can be done is to replace **..fail..** by **failwith** “EmptyList”, and use the following type specification

```
tail : List(A) → List(A)
```

This corresponds to an unchecked exception **unEx:UnExc** in our core language.

Using union types, we can also make use of either **err : Err** or **chEx : ChExc** to replace the **..fail..** clause. These two replacements result in the following two type specifications, respectively, for our **tail** method.

```
tail : req[xs] xs:List(A) ens[r] r:List(A) ∨ Err
tail : req[xs] xs:List(A) ens[r] r:List(A) ∨ ChExc
```

These two type-specifications state that either a runtime error value (of type **Err**) or a checked exception (of type **ChExc**) may be triggered, when **tail** method is invoked. Runtime error values (of type **Err**) may be passed in as arguments to functions, while checked exceptions (of type **ChExc**) can only be caught by try-catch constructs, and must be explicitly tracked in our framework. We discuss the details of exception handling in App C.2.

Note that we treat the **match** expression in **tail** function as type testing. Here, **Cons(.,.)** is just a subtype of **List** according to semantic subtyping. We can even write a more concrete case specification where data constructors are viewed as types. By making **..fail..** to be either **err**, **unExc** or **chExc**, we may use the following type specification.

```
tail : case[xs] { xs:Cons(T,List(T)) ⇒ ens[r] r:List(T)
                xs:Nil ⇒ ens[r] r:Err ∨ ChExc }
```

Once the verification procedures determine that the **tail** is called with a non-**Cons** input, the system can immediately figure out that it has encountered an error instead of leaving it to runtime. Lastly, if we were to use **abort** value for the **..fail..** clause, we could at best provide the following type-specification for the **tail** method, since **Abrt** type needs to be shown to be unreachable, meaning that there will not be any calls to **tail** with invalid arguments.


```
tail : req[xs] xs:Cons(_,List(T)) ens[r] r>List(T)
```

While the above type specification assumes that `xs` input is a non-empty `List(T)`-type, the `tail` function itself does not require such an assumption. Hence, it is even better to use a type specification with a weaker pre-condition, as follows.

```
tail : req[xs] xs:Cons(_,y) ens[r] r:y
```

C.2 Exception Handling

The `Abrt` or $\top$ types are nearly always avoided in the result type of our functions, as any encounter with abort will cause our program to fail immediately. Case specification is the only type specification which allows `Abrt` or $\top$ to appear in the `ens` clauses of the otherwise (i.e. `_`) clause, due to the need for completeness in case specifications.

Runtime errors are often modelled as exceptions that could be handled by suitable error recovery routines. In some languages, such as Java, they are further classified as *checked exceptions* which must be flagged by the type system or *unchecked exceptions* whose occurrences can be ignored by the type system.

In our solution, we would classify checked exceptions (denoted by `ChExc`) and unchecked exceptions (denoted by `UnExc`) as subtypes of `Exc`:

$$\perp <: \{\text{ChExc}, \text{UnExc}\} <: \text{Exc} <: \top$$

Additionally, `UnExc` is a subtype of valid types and `ChExc` is disjoint from both `Any` and `Abrt`:

$$\begin{aligned} \text{UnExc} &<: \{..valid types..\} \wedge \\ ((\text{ChExc} \wedge \text{Any}) &= (\text{ChExc} \wedge \text{Abrt}) = \perp) \end{aligned}$$

In this way, checked exceptions will always be carefully tracked by our type system, while unchecked exceptions are considered as a subtype of various valid types, and can be omitted in our type specifications. We recall this subtype relation in our lattice of types in Fig 1, where `..valid types..` include all standard types, e.g. `Int`, `Str`, `List(Int)`, and their separation counterparts.

Try-catch handling can now be implemented via a `let`-binding, followed by type testing. Raised exceptions that pass type-casting are propagated outwards via `match` construct, as illustrated by the second translation rule below.

$$\begin{aligned} &\text{try } e_1 \text{ catch Exc} \rightarrow e_2 \\ \Leftrightarrow &\text{let } x = e_1 \text{ in match } x \{ \text{Exc} \rightarrow e_2 \mid _ \rightarrow x \} \\ \\ &\text{let } x:t = e_1 \text{ in } e_2 \\ \Leftrightarrow &\text{let } x = (t \vee \text{Exc}) \text{ } e_1 \text{ in} \\ &\text{match } x \{ \text{Abrt} \vee \text{Exc} \rightarrow x \mid _ \rightarrow e_2 \} \end{aligned}$$

D More on Separation Types

D.1 Uniqueness vs Separation Type

One may also wonder how the uniqueness type is related to separation type. In our framework, each uniqueness type $x \mapsto T@U$ is (mutually) disjoint from both separation type $x \mapsto T$ and pure type $x : T$, as shown by the contradictions. Note that $\Leftrightarrow$ denotes logical equivalence.

$$x \mapsto T@U \wedge x : T \Leftrightarrow x \mapsto T \wedge x : T \Leftrightarrow x \mapsto T * x \mapsto T@U \Leftrightarrow \text{false}$$

This design decision is key towards better functionalities for each of the three types. However, we can rely on the following equivalence lemmas between uniqueness and separation types, which allow interconvertibility.

$$\begin{aligned} x \mapsto T@U &\Leftrightarrow x \mapsto T \wedge \text{NoAlias}(x) \\ \text{NoAlias}(x) \wedge x=y &\Leftrightarrow \text{false} \end{aligned}$$

These two equations state that every uniqueness type $x \mapsto T@U$ can always be converted to its corresponding separation type $x \mapsto T$; while each separation type can be converted to its uniqueness type if all aliases of form $x=y$ are dropped.

As type-safety is essentially a static analysis based on over-approximation, we support the following weakening lemmas (that can be performed safely):

$$\begin{aligned} x \mapsto T@U \wedge T <: A &\Rightarrow_{\text{weaken}} x \mapsto A@U \\ x \mapsto T \wedge T <: A &\Rightarrow_{\text{weaken}} x \mapsto A \\ x \mapsto T &\Rightarrow_{\text{weaken}} x : T \end{aligned}$$

We thus always start with types that are as strong as possible, but allow them to be weakened, when necessary. We also start with separation/uniqueness types when the data structures are first constructed, but allow them to be irreversibly weakened to flow-insensitive pure-type, as shown in the last lemma above.

Both uniqueness and separation types can be mutated in support of flow-sensitivity. However, the separation type also allows must-aliases, while the uniqueness type allows only a single reference. Hence, separation type is strictly more expressive. To illustrate the need for aliasing, consider the earlier `swap` method to exchange the contents of two heap locations, namely:

```
swap(x, y) = let v1=!x in let v2=!y in
             update(x, v2); update(y, v1)
```

Uniqueness type's solution cannot currently handle `swap(m, m)` since the two parameter objects must be unaliased.

D.2 Pure Predicate vs Separation Predicate

In strongly-typed languages, algebraic data types are typically declared. Two simple examples are shown below.

```

data Ref(B)   = Ref(B)
data List(T)  = Nil | Cons(T, List(T))

```

where the `Ref` is a non-recursive type, while `List` is a recursive type. Note that `Ref(B)` in the RHS is overloaded to denote a data constructor.

To represent flow-insensitive types for these data structures, we can use the following pure predicates, where the latter is an inductive pure predicate.

```

x : Ref(B : AnyP)  <=>  ex b : AnyP. x : Ref(b) /\ b : B
x : List(B : AnyP) <=>  x : Nil  \/  ex b, q : AnyP . x : Cons(b, y)
                                     /\ b : B /\ q : List(B)

```

For separation predicates, we are primarily interested in capturing values, rather than types. Thus, we will make use of `B : Any` (as arguments to separation predicate) to denote either a pure type or a separation type.

With this difference, we can define the *default* separation predicate for both `Ref` and `List` types, as follows:

```

x -> Ref(B : Any)  <=>  ex b : Any. Ref(b) * b : B
x -> List(B : Any) <=>  x : Nil  \/  ex b, q : Any . x -> Cons(b, y)
                                     * b : B * q -> List(B)

```

Here, `b : Any` denotes either a pure type or a separation type, depending on what is being supplied as `B`. For example, `x -> Ref(Ref(Int))` would contain pure argument `B = Ref(Int)` denoting $\exists b : \text{Any}. x \mapsto \text{Ref}(b) \wedge b : \text{Ref}(\text{Int})$. In contrast, the type `x -> Ref( $\mapsto \text{Ref}(\text{Int})$ )` would denote $\exists b : \text{Any}. x \mapsto \text{Ref}(b) * b \mapsto \text{Ref}(\text{Int})$.

In the case of `List`, the above separation predicate actually captures an acyclic list that will eventually terminate with `Nil`. It is also possible to construct other kinds of separation predicates for `List`. Another example, namely circular `List`, is described in App D.3. Such finely differentiated data structures are only possible with separation type predicates, and not expressible using pure type predicates since the former has full ownership of its memory, while the pure flow-insensitive predicates do not (as their data structures are arbitrarily aliased). In particular, pure predicates are required to be stable under heap mutation.

Note that each default declaration of types, such as `x : List(t)` or `r : Ref(t)`, is stable, and we can always use its nested form, such e.g. `x : List(List(t))` without losing the stability of pure types. In addition, for pass-by-value parameters, we can use its outermost data constructors as its stable pure type, e.g. `x : Nil` or `x : Cons(T, List(T))` are stable, since the reference to the outermost constructor would have been copied during the call and would be unchanged by other aliases. If we had immutable data constructors, we can also use more deeply nested constructors, such as `x : Cons(T, Nil)`, since these data constructors' fields are immutable (and stable). However, for mutable `Ref(T)` type where its field `T` is mutable, we cannot use `r : Ref(Nil)` or `r : Ref(Cons(Int, List(Int)))`. Thus, each mutable field (or pass-by-reference parameter) could only take its default (stable) pure type, and not use data constructors as its type argument, since the references of mutable field may be changed via other aliases. Compared to pure types, separation types are always stable. As an example, consider:

```
match !x of { Nil -> 1;
             Cons(_,_) -> "hi" }
```

Using pure type, we can at best write the following type-specification.

```
req[x] x:Ref(List(_)) ens[r] r:Int\Str
```

Using separation type, we can write a more precise type-specification.

```
case[x]{ x->Ref(Nil) ens[r] r:Int;
        x->Ref(Cons(_,_)) ens[r] r:Str }
```

D.3 Separation Type for Data Structure Invariance

For pure types, one typically uses inductive predicates to capture the weakest property that is always true for each given algebraic data type. As an example, consider the `List(T)` algebraic data type that is defined, as follows:

```
data List(T) = Nil | Cons(T, List(T))
```

The weakest predicate for such a `List(T)` data type would be:

$$r:\text{List}(T) \equiv r:\text{Nil} \vee (\exists q. r:\text{Cons}(T, q) \wedge q:\text{List}(T))$$

This predicate is stable for every pure `List(T)` type that can be (mutably) built from such a data type. Let us consider two `List(T)` examples, namely one *acyclic* and the other *cyclic*, as shown below. Both of these examples can be proven to be instances of the `List(T)` predicate type, as shown below.

```
// acyclic list is an instance of x:List(T)
x:Cons(T,y)/\y:Cons(T,z)/\z:Nil |- x:List(T)

// non-empty cyclic list is also an instance of List(T)
x:Cons(T,y)/\y:Cons(T,z)/\z:x |- x:List(T)
```

Using separation type, we can distinguish between a (possibly empty) acyclic list and a (non-empty) cyclic list by the following set of separation predicates.

```
x->AList(T)  <=> x:Nil /\ ex q. x->Cons(T,q)*q->AList(T)
x->CList(T)  <=> x->Cons(T,q)*q->LSeg(T,z)
x->LSeg(T,p) <=> x:p /\ ex q. x->Cons(T,q)*q->LSeg(T)
                    /\x:¬p
```

To support circular list, one would need to have list-segment predicate, $x \mapsto \text{LSeg}(T, p)$, which captures an incoming pointer x and an outgoing pointer p .

Using these separation predicates, we can prove the following examples:

```
x->Cons(T,y) * y->Cons(T,z) /\ z:Nil |- x->AList(T)
x->Cons(T,y) * y->Cons(T,z) /\ z:x   |- x->CList(T)
```

But may also disprove:

```
x->Cons(T,y) * y->Cons(T,z) /\ z:Nil |/- x->CList(T)
x->Cons(T,y) * y->Cons(T,z) /\ z:x   |/- x->AList(T)
```

Thus, separation predicates are flow-sensitive, and they typically denote *data structure invariance* that we may need to guarantee for our program codes. We propose to provide this guarantee by typed-safe verification using pre/post conditions, expressed with the help of suitable separation predicates.

D.4 Automatic Weakening of Separation to Pure Type

The weakening lemma allows each separation type to be irreversibly weakened to its generalized pure type, as shown below.

$$\mathbf{x} \mapsto \mathbf{T} \wedge \mathbf{T} <: \mathbf{Any}_P \wedge \mathbf{T} <: \mathbf{A} \Rightarrow_{\text{weaken}} \mathbf{x} : \mathbf{A}$$

Each pure type used $\mathbf{x} : \mathbf{A}$ must be stable under the allowable heap mutation, so that it can be used as a flow-insensitive type. Moreover, we can perform this lemma weakening automatically, on demand. This is achieved using the following entailment rule, which irreversibly weakens Δ to $\Delta_1 \wedge \mathbf{x} : \mathbf{t}$

$$\frac{\text{stable}(\mathbf{x} : \mathbf{t}) \quad \Delta \vdash (\mathbf{x} \mapsto \mathbf{t} \wedge \mathbf{t} <: \mathbf{Any}_P) \Rightarrow \Delta_1}{\Delta \vdash \mathbf{x} : \mathbf{t} \Rightarrow \Delta_1 \wedge \mathbf{x} : \mathbf{t}}$$

D.5 Separation Type for Data Race Freedom

A concurrent program of two threads has a race if both threads access the same shared data, and one of them is a write; and there is no synchronization order between these accesses of the two threads. One conservative way to ensure race freedom between two threads is to guarantee that they do not have shared memory, where one thread has full ownership (to write) and the other thread has at least shared ownership to read. Such ownership can be denoted by separation types, but we will need finer granularity of ownership on a per-field basis.

Consider a mutable `Pair(X, Y)` with two fields of types `X` and `Y`.

```
data Pair(X, Y) = Pair(X, Y)
```

and the following two concurrent threads.

```
p.Pair.1 := "hi" || p.Pair.2 := p.Pair.2+1
```

Though both threads have write access to `p`, they are really operating on disjoint fields. To support such access, we shall provide the following lemma for each object type which can be used to convert to field-level separation types.

$$\mathbf{p} \rightarrow \text{Pair}(\mathbf{X}, \mathbf{Y}) \Leftrightarrow \mathbf{p}.\text{Pair}.1 \rightarrow \mathbf{X} * \mathbf{p}.\text{Pair}.2 \rightarrow \mathbf{Y}$$

Thus, one can denote each `Pair` object by the separation type $\mathbf{p} \rightarrow \text{Pair}(\mathbf{X}, \mathbf{Y})$ which can be split into two fields $\mathbf{p}.\text{Pair}.1 \rightarrow \mathbf{X} * \mathbf{p}.\text{Pair}.2 \rightarrow \mathbf{Y}$ by the above lemma.

To ensure data-race freedom, we can only use separation types in concurrent threads, but must not use pure types (unless we are using immutable data structures), since the latter can be mutated by other threads via (arbitrary) aliases of pure types. Type-safe verification of our example can be performed, as follows.

```

// p->Pair(Any, Int)
// p.Pair.1->Any * p.Pair.2->Int
p.Pair.1 := "hi" || p.Pair.2 := p.Pair.2+1
// p.Pair.1->Str * p.Pair.2->Int
// p->Pair(Str, Int)

```

In case concurrent threads require write-accesses to shared heap memory, we can further extend our framework to support a locking mechanism that can be applied to data structures that are captured by separation types.

E Operational Semantics

To facilitate the following soundness proofs, we define a big-step reduction relation with judgments $e, h, S \rightsquigarrow R_e, h_1, S_1$. Program states consist of a heap h and store S , like in Section 5. Outcomes are $R_e ::= \text{Norm}(v) \mid \mathbf{Abt}$ where $\text{Norm}(v)$ captures the valid values and $\mathbf{Err}$ values which could occur during the run-time but do not terminate the programs. The $\mathbf{Abt}$ represents a failure which will immediately kill the process. In the *Match* case, we assume there is an implicit case that handles a match failure. If the process cannot find a case to match, it will terminate the program with $\mathbf{Abt}$.

$$\begin{array}{c}
\frac{}{v, h, S \rightsquigarrow v, h, S} \text{Nil, Const, Lambda} \\
\\
\frac{}{x_1 :: x_2, h, S \rightsquigarrow S(x_1) :: S(x_2), h, S} \text{Cons} \\
\\
\frac{}{x, h, S \rightsquigarrow S(x), h, S} \text{Var} \\
\\
\frac{\text{type}(S(x)) <: t}{(t(x)), h, S \rightsquigarrow x, h, S} \text{Cast} \\
\\
\frac{\text{type}(S(x)) \not<: t}{(t(x)), h, S \rightsquigarrow \mathbf{Abt}, h, S} \text{Cast} \\
\\
\frac{e_1, h, S \rightsquigarrow v, h_1, S_1 \quad e_2, h_1, S_1[x := v] \rightsquigarrow v_1, h_2, S_2}{(\text{let } x = e_1 \text{ in } e_2), h, S \rightsquigarrow v_1, h_2, S_2 \setminus \{x\}} \text{Let} \\
\\
\frac{S(f) = \lambda x^*. \Phi e \quad S(x^*) = v^* \quad [x^* := v^*]e, h, S \rightsquigarrow v_2, h_1, S_1}{f(x^*), h, S \rightsquigarrow v_2, h_1, S_1} \text{App} \\
\\
\frac{x_1 \in \text{dom}(h)}{(x_1 := x_2), h, S \rightsquigarrow (), h_1[S(x) := S(v)], S} \text{Assign} \\
\\
\frac{}{!x, h, S \rightsquigarrow h(S(x)), h, S} \text{Deref} \\
\\
\frac{l \notin \text{dom}(h)}{\text{ref } x, h, S \rightsquigarrow \ell, h[\ell := S(x)], S} \text{Ref}
\end{array}$$

$$\frac{S(x) \subseteq \mathbf{p_i} \quad e_i, h, s \rightsquigarrow v, h_i, S_i}{(\mathbf{match} \ x \ \{ \mathbf{p_1} \rightarrow \mathbf{e_1} \mid \dots \mid \mathbf{p_n} \rightarrow \mathbf{e_n} \}), h, S \rightsquigarrow v, h_i, S_i} \textit{Match}$$